LeetCode 102 — Binary Tree Level Order Traversal

🔗 https://leetcode.com/problems/binary-tree-level-order-traversal/

1. Problem Title & Link

Title: LeetCode 102: Binary Tree Level Order Traversal
Link: https://leetcode.com/problems/binary-tree-level-order-traversal/

2. Problem Statement — Simple Summary

Return the **level-by-level traversal** of the binary tree.

Visit nodes:

Level 1 → Level 2 → Level 3 → ...

(left → right per level)

Output as a **list of lists**.

3. Examples

Example 1

Output: [[3],[9,20],[15,7]]

Example 2

Input: []
Output: []

Example 3

Input:

1

\

2

Output: [[1],[2]]

4. Constraints

- 0 ≤ #nodes ≤ 2000
- Standard BFS level order
- Use queue

5. Core Concept / Pattern

⭐ **Breadth First Search (BFS) using Queue**

Traverse each depth level fully before moving to next.

6. Thought Process — Step-by-Step

1. Push root into queue
2. For each level:
    - Get queue size → number of nodes in that level
    - Pop exactly those many nodes
    - Add children of each popped node back to queue
3. Append one level result each loop

This guarantees **perfect level grouping**.

7. Visual Diagram

Tree:

```
      3
     / \
    9  20
       / \
      15  7
```

BFS Queue movement:

[3] → pop → [9,20]

[9,20] → pop → [15,7]

[15,7] → pop → []

Result building:

[[3],[9,20],[15,7]]

8. Pseudocode

```
if root == null → return []

queue = [root]
result = []

while queue not empty:
    levelSize = queue.size
    levelList = []
```

```
    repeat levelSize times:
        node = queue.pop_front
        levelList.append(node.val)
        if node.left  exists → queue.push(left)
        if node.right exists → queue.push(right)

    add levelList to result

return result
```

9. Code Implementation

✔ **Python**

```python
from collections import deque

class Solution:
    def levelOrder(self, root):
        if not root: return []

        q = deque([root])
        res = []

        while q:
            size = len(q)
            level = []

            for _ in range(size):
                node = q.popleft()
                level.append(node.val)

                if node.left: q.append(node.left)
                if node.right: q.append(node.right)

            res.append(level)

        return res
```

✔ **Java**

```java
class Solution {
    public List<List<Integer>> levelOrder(TreeNode root) {
        List<List<Integer>> res = new ArrayList<>();
        if (root == null) return res;
```

```
        Queue<TreeNode> q = new LinkedList<>();
        q.offer(root);

        while (!q.isEmpty()) {
            int size = q.size();
            List<Integer> level = new ArrayList<>();

            for (int i = 0; i < size; i++) {
                TreeNode node = q.poll();
                level.add(node.val);

                if (node.left != null)  q.offer(node.left);
                if (node.right != null) q.offer(node.right);
            }
            res.add(level);
        }
        return res;
    }
}
```

10. Time & Space Complexity

Operation	Complexity
Time	**O(n)** — every node visited once
Space	**O(n)** — queue holds nodes of largest level

11. Common Mistakes / Edge Cases

❌ Forgetting to push children

❌ Adding values directly — mixing levels

❌ Using DFS but grouping wrong

Edge cases:

✔ Empty tree → return []

✔ Only root → [[root]]

✔ Skewed tree → many levels with one node

12. Dry Run

Tree:

```
  1
 / \
2   3
     \
      4
```

Queue	Level Output
[1]	[1]
[2,3]	[2,3]
[4]	[4]
[]	End

Final → [[1],[2,3],[4]]

13. Real-Life Use Cases

- Shortest path in unweighted graph
- Traversing hierarchical file structures
- Organization charts / Family tree
- Networking broadcast simulation

14. Traps

⚠ Using stack (gives DFS, wrong order)

⚠ Not storing level size before processing

⚠ Appending values directly to main list

15. Builds To Related Problems

Problem	Extension
LC 107	Reverse level order
LC 103	Zigzag level order
LC 199	Right side view
LC 102 → 515 → 637	Max per level / average

16. Alternate Techniques

Method	When
BFS Queue ⭐	Default & best

DFS (add depth manually)	Good for recursion lovers

17. Why This Works

We process **one depth fully** before going to next — BFS naturally produces level order.

18. Follow-Up

- Zigzag output? (Alternate left-right per row)
- Return level sums, max, min instead of lists
- Return tree by depth grouping (like adjacency levels)